

Sistemas Operativos 2026



Clase 3
U2 Hilos

Ing. Juan Manuel Calvo

Jerarquía de los procesos

- UNIX un solo arbol, init o systemd, arranca las terminales y espera que un usuario se conecte.
- Windows, no existe una jeraquía, cuando un proceso crea a otro, el primero recibe un handle que le permite controlar al creado.
- Ej Linux
\$ pstree

Como listar los procesos

- En Linux se pueden usar estos comandos ps ó top
 - \$ ps fax
 - \$ top
- En Windows el Task Manager

Obtener información de los procesos

- En Linux
 - \$ cat /proc/self/status
- ó
- \$ cat /proc/nnn/status donde nnn es el número de proceso

Finalizar un proceso

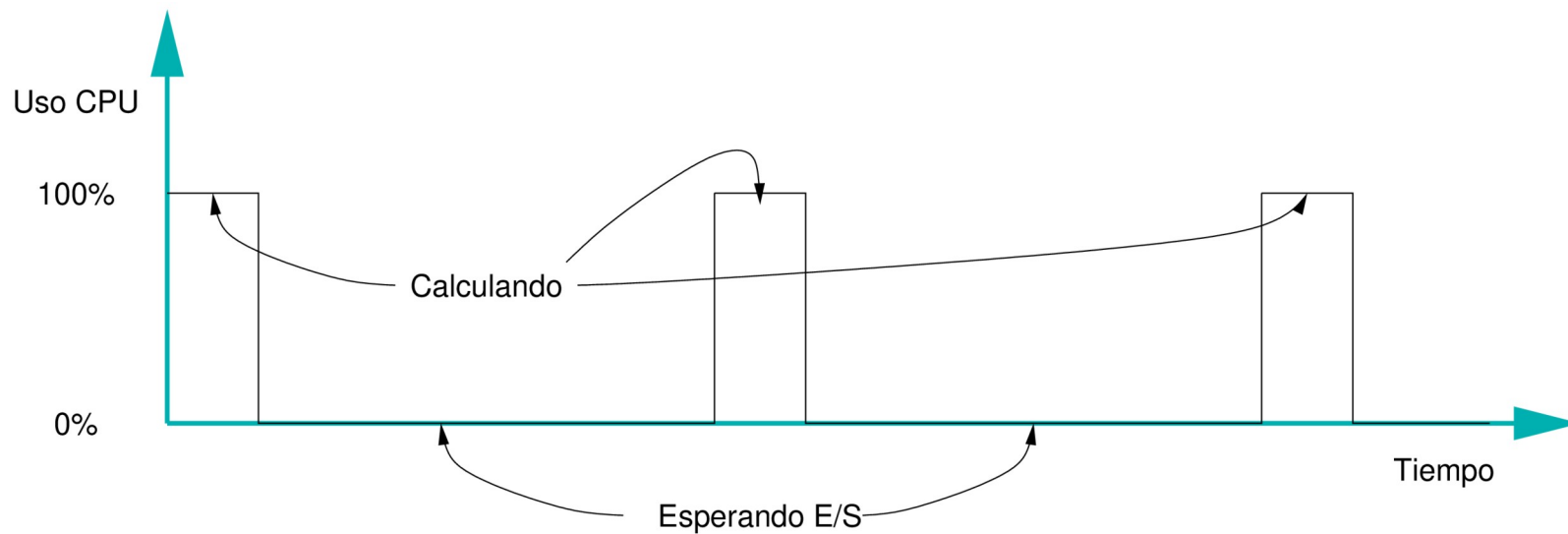
```
xclock -update 1 &  
...  
$ (sleep 20;echo fin) &  
...  
$ kill nnn
```



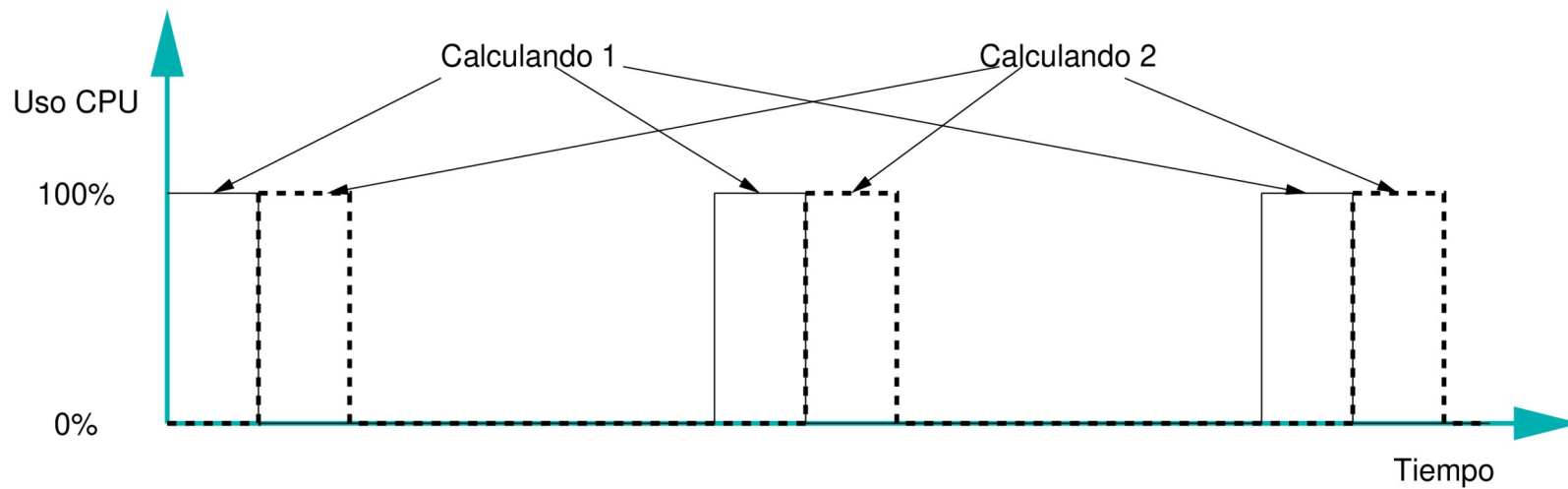
UNIVERSIDAD
DEL CEMA

UCEMA

Uso de la CPU por un proceso

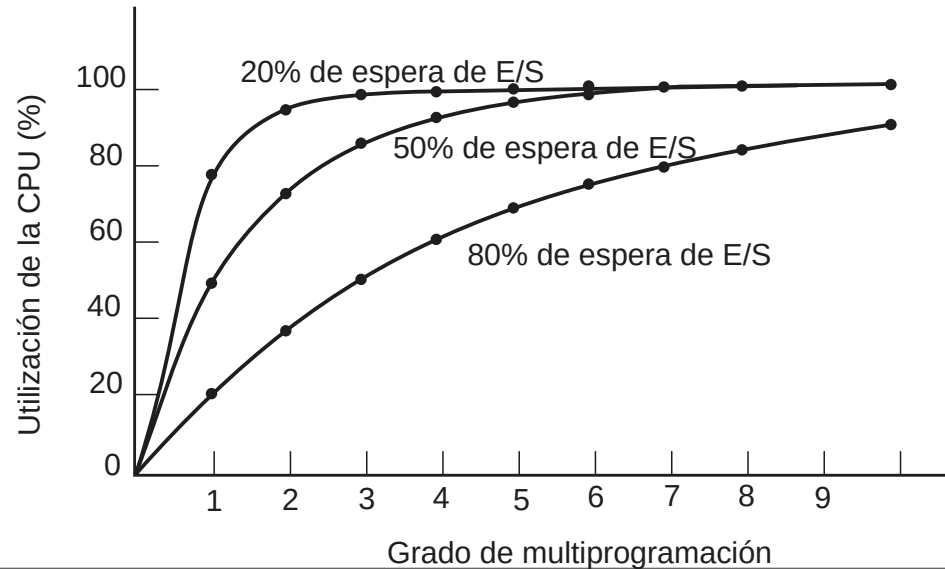


Uso de la CPU por dos procesos

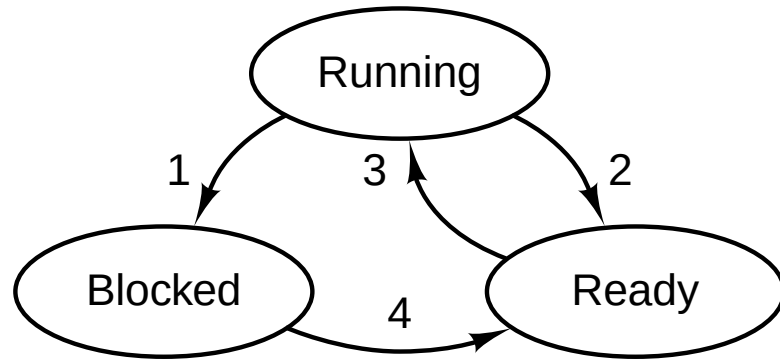


Modelado multiprogramación

- Suponga que un proceso gasta una fracción p de su tiempo esperando que se complete la E/S.
- Con n procesos en la memoria a la vez, la probabilidad de que todos los n procesos estén esperando E/S (en cuyo caso la CPU estará inactiva) es p^n .
- La utilización de la CPU viene dada por la fórmula $\Rightarrow \text{CPU utilization} = 1 - p^n$



Estado de los procesos



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

- ¿Es posible un sistema donde no exista la transición 2?

Cooperative multitasking

Implementación de los procesos

- Tabla de procesos (Process control blocks PCB), incluye:
 - Contador de programa
 - Stack pointer
 - Asignación de memoria
 - Estado de los archivos abiertos
 - Y todo lo demás que debe ser guardado cuando se interrumpe el proceso
 - De manera que pueda ser reiniciado más tarde
 - Como si nunca se hubiera detenido

Xv6 process control blocks

```
// Per-process state
struct proc {
    struct spinlock lock;
    enum procstate state; // Process state
    void *chan;           // If non-zero, sleeping on chan
    int killed;           // If non-zero, have been killed
    int xstate;           // Exit status to be returned to parent's wait
    int pid;              // Process ID
    struct proc *parent; // Parent process
    uint64 kstack;        // Virtual address of kernel stack
    uint64 sz;            // Size of process memory (bytes)
    pagetable_t pagetable; // User page table
    struct trapframe *trapframe; // data page for trampoline.S
    struct context context; // switch() here to run process
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;        // Current directory
    char name[16];           // Process name (debugging)
    long ctx_switches;       // Number of context switches
};
```

Xv6 espacio para guardar el estado interno del procesador

```
struct trapframe {  
    /* 0 */ uint64 kernel_satp; // kernel page table          /* 144 */ uint64 a4;  
    /* 8 */ uint64 kernel_sp; // top of process's kernel stack /* 152 */ uint64 a5;  
    /* 16 */ uint64 kernel_trap; // usertrap()                 /* 160 */ uint64 a6;  
    /* 24 */ uint64 epc; // saved user program counter         /* 168 */ uint64 a7;  
    /* 32 */ uint64 kernel_hartid; // saved kernel tp          /* 176 */ uint64 s2;  
    /* 40 */ uint64 ra; // saved register ra                    /* 184 */ uint64 s3;  
    /* 48 */ uint64 sp; // saved register sp                    /* 192 */ uint64 s4;  
    /* 56 */ uint64 gp; // saved register gp                    /* 200 */ uint64 s5;  
    /* 64 */ uint64 tp; // saved register tp                    /* 208 */ uint64 s6;  
    /* 72 */ uint64 t0; // saved register t0                    /* 216 */ uint64 s7;  
    /* 80 */ uint64 t1; // saved register t1                    /* 224 */ uint64 s8;  
    /* 88 */ uint64 t2; // saved register t2                    /* 232 */ uint64 s9;  
    /* 96 */ uint64 s0; // saved register s0                    /* 240 */ uint64 s10;  
    /* 104 */ uint64 s1; // saved register s1                   /* 248 */ uint64 s11;  
    /* 112 */ uint64 a0; // saved register a0                   /* 256 */ uint64 t3;  
    /* 120 */ uint64 a1; // saved register a1                   /* 264 */ uint64 t4;  
    /* 128 */ uint64 a2; // saved register a2                   /* 272 */ uint64 t5;  
    /* 136 */ uint64 a3; // saved register a3                   /* 280 */ uint64 t6;  
  
};
```

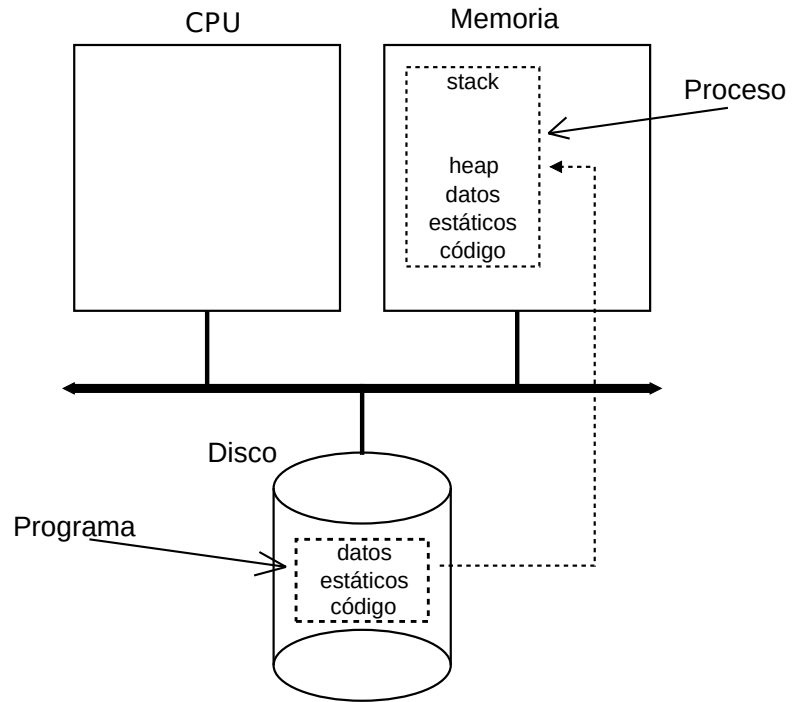
Implementación de los procesos



Manejo de procesos	Manejo de memoria	Manejo de archivos
Contador de programa Stack pointer Otros registros Estado del proceso Prioridad Parámetros de planificación Identificador del proceso Proceso padre Grupo de procesos Señales Tiempo cuando el proceso arrancó Tiempo usado de CPU	Punteros para definir los segmentos de código, datos y stack	Directorio raíz Directorio de trabajo Descriptores de archivo Identificador de usuario y grupo

Algunos de los campos de una entrada de la tabla de procesos

Programa y proceso



En resumen ¿que es un proceso?

- Un programa ejecutándose
 - un espacio de direcciones de memoria (conteniendo código y datos)
 - varios otros recursos (Ej: archivos abiertos)
 - informacion de estado (valor del contador de programa, los registros, el puntero a la pila, etc)
 - todo lo guardado en el PCB del proceso

En resumen ¿que es un proceso? II

- Es realmente dos categorías de cosas
 - una colección de recursos, el código y el espacio de direcciones, los archivos abiertos, etc.
 - un hilo de ejecución, el estado actual que opera sobre esos recursos
- podemos pensar esas dos cosas separadamente

Hilos en el modelo de procesos

- Gran parte del trabajo del sistema operativo tiene el objetivo evitar que los procesos interfieran unos con otros.
 - Cada hilo de ejecución esté asociado con su propia agrupación de recursos (proceso).
 - Que no se pueda escribir sobre el espacio de direcciones de otro proceso.
- Esto es correcto, pero significa que si Ud. quiere cambiar de hilo necesita cambiar de proceso.
 - Requiere la intervención del sistema operativo y un costoso cambio de contexto.
 - esto es una lástima dado que algunas aplicaciones consisten logicamente en más de un hilo pero solo necesitan de una única agrupación de recursos.

Aplicaciones multihilo

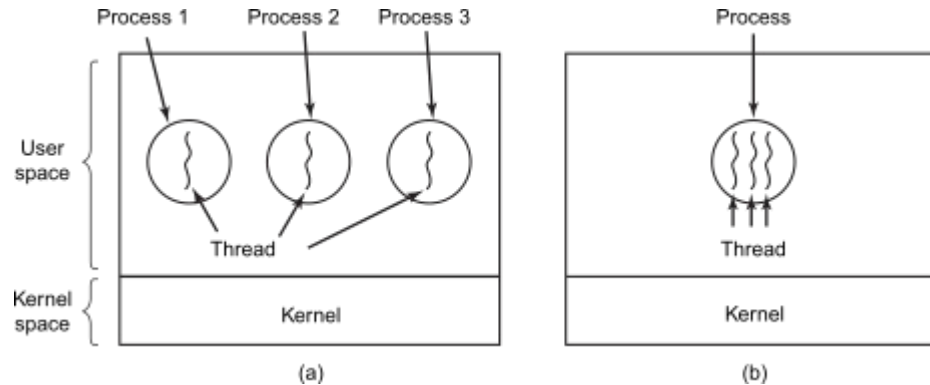
- ¿Podemos permitir a múltiples hilos de ejecución compartir un espacio de direcciones de memoria común?
 - para aplicaciones con:
 - la necesidad de tener estructuras de datos en común entre los hilos.
 - no haya necesidad que el sistema operativo haga respetar la separación de recursos (los hilos deben confiar unos de otros).
 - no es adecuado para código arbitrario/programas en general.
- Aplicaciones ejemplo:
 - web server, sirve a varios clientes a la vez
 - web browser, carga diferentes páginas simultáneamente
 - Editor de texto, un hilo almacena las teclas, otro redibuja la pantalla, otro hace el autoguardado, etc.
-

Tres maneras de hacer un servidor

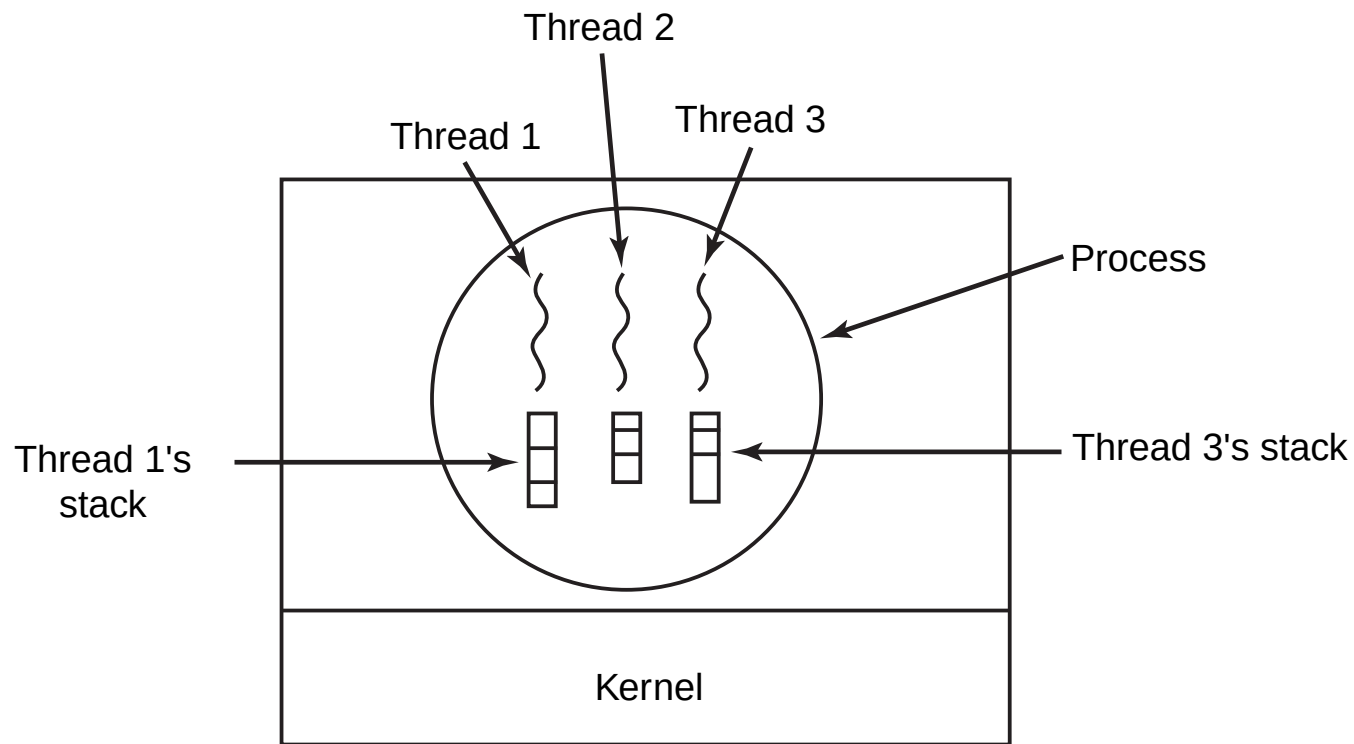
Modelo	Características
Proceso de un sólo hilo	Sin paralelismo, llamadas al sistema bloqueantes
Hilos	Paralelismo, llamadas al sistema bloqueantes
Máquina de estados finitos	Paralelismo, llamadas al sistema no bloqueantes, interrupciones

Modelo clásico de hilos

Ítems por proceso	Ítems por hilo
Espacio de direcciones Variables globales Archivos abiertos Procesos hijos Alarmas pendientes Señales y manejadores de señal Información contable	Contador de programa Registros Stack Estado



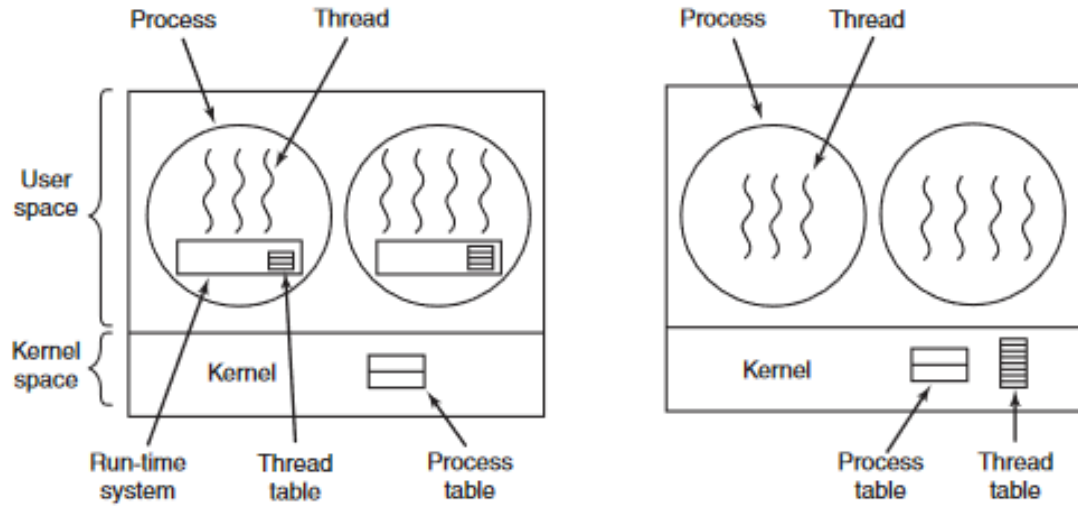
Cada hilo tiene su propio stack



UNIVERSIDAD
DEL CEMA

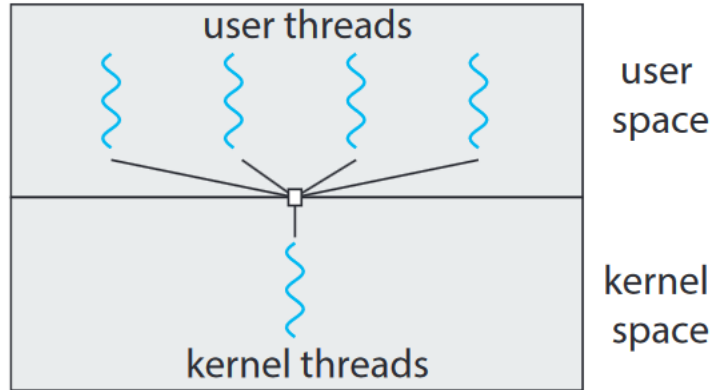
UCEMA

Implementación de hilos en espacio del usuario y en el kernel



Modelos de multihilo: Many-to-One

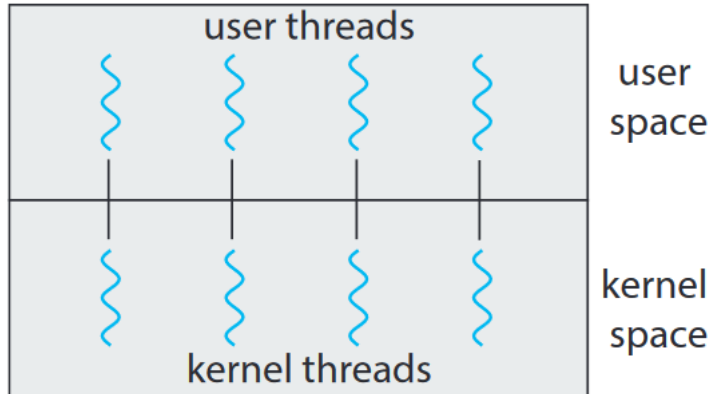
Modelo muchos-a-uno



- El manejo de los hilos se hace por una biblioteca en el espacio del usuario
 - Es eficiente
 - Pero todo el proceso se bloquea si un hilo hace una llamada bloqueante
- En un dado momento solo un hilo puede correr en el kernel
 - No es posible usar varios núcleos en un sistema multinúcleo
- Usado por la biblioteca **Green Threads** en las primeras implementaciones de Java, actualmente poco usado.

Modelos de multihilo: One-to-One

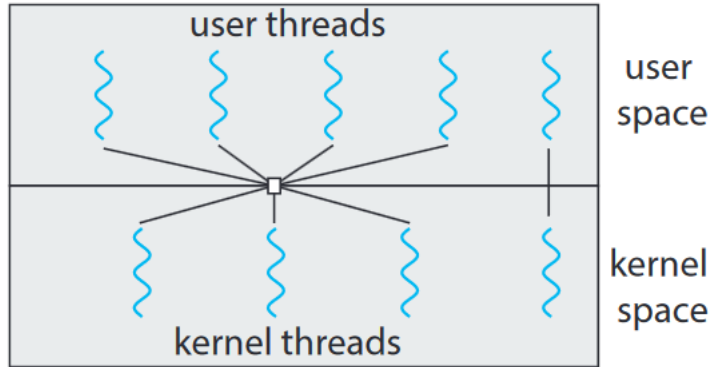
Modelo uno-a-uno



- Permite que cuando un hilo hace una llamada bloqueante otro hilo siga corriendo.
- Permite que varios hilos se ejecuten concurrentemente en multiprocesadores
- Requiere que cada vez que se crea un hilo de usuario se cree el correspondiente hilo en el kernel.
 - Muchos hilos en el kernel pueden sobrecargar el sistema.
- Modelo usado en Linux y Windows

Modelos de multihilo: Many-to-Many

Modelo Muchos-a-muchos



- Hilos en el kernel adecuados a los recursos de hardware
- Modelo más flexible, pero difícil de implementar

Hilos POSIX

Thread call	Descripción
pthread_create	Crear un nuevo hilo
pthread_exit	Termina el hilo llamante
pthread_join	Esperar que un hilo específico termine
pthread_yield	Liberar la CPU para que corra otro hilo
pthread_attr_init	Crear e inicializar la estructura de atributos de un hilo
pthread_attr_destroy	Borrar la estructura de atributos del hilo

Programa ejemplo usando hilos



```
/* compilar con
 * gcc -Wall -pthread -o thread_ex thread_ex.c
 */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS 10

void *print_hello_world(void *tid)
{
    /* This function prints the thread's identifier and then exits.
    */
    printf("Hello World. Greetings from thread %ld\n", (long) tid);
    pthread_exit(0);
}
```

```
int main(int argc, char *argv[])
{
    /* The main program creates 10 threads and then exits. */
    pthread_t threads[NUMBER_OF_THREADS];
    int status;
    long i;
    for(i=1; i <= NUMBER_OF_THREADS; i++) {
        printf("Main here. Creating thread %ld\n", i);
        status = pthread_create(&threads[i], NULL,
        print_hello_world, (void *)i);
        if (status != 0) {
            printf("Oops. pthread create returned error code %d\n",
            status);
            exit(-1);
        }
    }
    exit(0);
}
```